

プログラム説明資料（まとめ版）

石川原 吉美 / 株式会社ラセングル 二次面接（2026年8月18日 14:30～ 対面）持参用

この資料は、これまでに制作した6作品の技術的なポイントをA4数枚にまとめたものです。次ページ以降で、特に技術的な作り込みが多い3作品（MACHINA PARTY／BOMBER ARENA／FlyingBird）を1作品1ページで詳しく紹介します。それ以外の3作品についても、下の一覧の内容でご質問にお答えできます。プレイ動画はその場でiPadからご覧いただけます。

作品名	ジャンル・タグ	開発体制・期間	担当範囲・アピール
MACHINA PARTY	3Dアクション／マルチプレイ・パーティー ※制作中	2人体制（本人＋1名） 2026年4月～	ゲーム内容全般（GameManager・Tank関連システム・レースシステム等） → 次ページで詳細
BOMBER ARENA	2Dアクション／マルチプレイ対戦	個人開発（1名） 2025年12月～2026年2月	企画・設計・実装・デバッグ・UI制作まで全工程 → 次ページで詳細
FlyingBird	2Dカジュアルアクション	個人開発（1名） 2025年7月～8月頃	企画・設計・実装・デバッグ・サウンド組込みまで全工程 → 次ページで詳細
避けるんデス	2D弾幕シューティング／チーム制作	Prog×3 2025年6月～8月	プレイヤー制御・アイテムシステム・ゲーム進行管理・入力管理。新InputSystemでコントローラーに対応し、デバイスに依存しない入力設計。
ケガレの館	2Dホラー脱出／ゲームジャム（3日間）	Prog×4 Des×1 1年時・3日間制作	プレイヤー操作・ダメージ処理・当たり判定。ダメージ処理を共通化し、敵やトラップの追加に対応しやすい設計。
XEVIOUS	2Dシューティング／アーケード版の再現・チーム制作	Prog×4 2024年10月～2025年2月	自機システム全般（移動・弾・残機・当たり判定・リスポーン・マップスクロール）。Inspectorから調整可能な拡張性の高い設計。

GitHubリンク・プレイ動画（YouTube）は各作品ページ、およびポートフォリオサイトに掲載しています。

ジャンル マルチプレイ3Dアクション (バトル+レースのパーティーゲーム) 開発環境 Unity 6 / C#

開発期間 2026年4月～ (制作中/週3～4回×2～3時間ペー ス) チーム規模 2人体制 (本人+1名)

担当範囲 ゲーム内容全般 (GameManager/Tank関連システム/レースシステム等のプログラム実装)

アピールポイント

GameManager設計

ラウンド制御・生存判定・勝者判定を1クラスに集約

TankShooting (射撃)

長押しチャージ+放物線計算で着弾を予測。特殊弾システムも搭載

TankAI (CPU制御)

NavMeshでSeek追跡⇄Flee逃走を自動遷移する自律AI

コンポーネント分離設計

TankManagerで管理を一元化し、再利用しやすい設計に

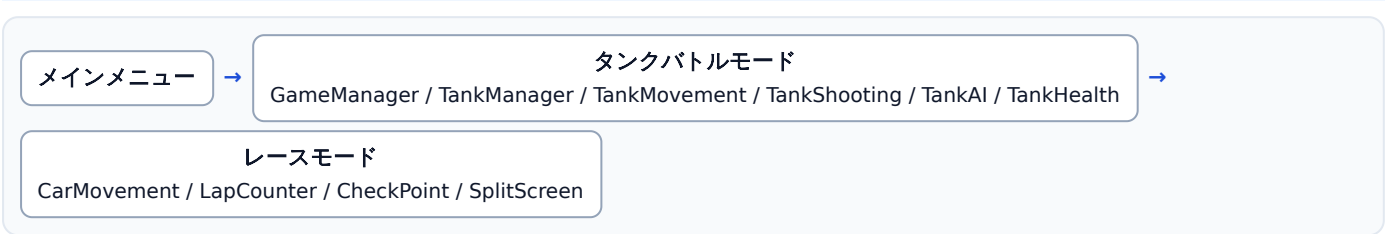
技術ハイライト

TankShooting: 放物線の二次方程式で着弾予測位置を算出し、AIの狙い撃ちにも活用

TankAI: NavMeshでSeek⇄Flee状態を距離・時間条件で自動遷移させる自律AI。経路探索間隔もランダム化して負荷分散

レースシステム: チェックポイントは順番通過のみ有効化して逆走・飛ばしを防止。人数に応じて分割画面カメラを自動生成

全体構成 (アーキテクチャ)



実装時間目安: 約100時間 (13週間・週3～4回×2～3時間)

GitHub: github.com/Yoshinko45/MACHINA_PARTY

動画: youtube.com/watch?v=kyzpTByk9C8

ジャンル マルチプレイ対戦アクション（爆弾を使ったバトル） 開発環境 Unity / C#

開発期間 2025年12月～2026年2月（約12週間） チーム規模 個人開発（1名）

担当範囲 企画・設計・プログラム実装・デバッグ・UI制作まで全工程

アピールポイント

対戦システムの統合設計

勝敗判定・ラウンド進行・生存管理を統合した対戦システム

爆弾～爆風～破壊の一連実装

設置～爆風展開～障害物破壊までの一連のシステムを実装

完全なステージリセット

ラウンド終了後、痕跡を残さずステージを完全復元

static設計

人数・操作方法・試合結果をstaticクラスでシーン間に保持

技術ハイライト

MatchManager : 演出中はTime.timeScale=0にしつつWaitForSecondsRealtimeでタイマーだけ進行させ、InputLockedで全プレイヤーの入力を一括ロック

BombController : 爆風の伝播をExplode()の再帰呼び出しで実装。OverlapBoxで壁／ブロックを判定し、壁なら停止・ブロックなら破壊して停止と分岐処理

StageResetter : ラウンド終了ごとにTilemapを保存データから完全復元し、生成物（爆弾・爆風・アイテム）を一括Destroyして残留物ゼロに

試合の流れ

タイトルで人数・CPU・操作を確定

ラウンド演出
入力ロック中

ゲーム進行
爆弾設置・アイテム取得

生存判定・勝敗

3勝未満：次ラウンドへ／3勝：結果画面

総開発工数目安：約80～90時間（12週間・週3回×2～3時間）

GitHub: github.com/Yoshinko45/Bomber-Arena

動画: youtube.com/watch?v=KcYYJUvRcik

ジャンル 2Dカジュアルアクション（フラッピーバード系）

開発環境 Unity / C#

開発期間 2025年7月～8月頃（約6～7週間）

チーム規模 個人開発（1名）

担当範囲 企画・設計・プログラム実装・デバッグ・サウンド組込みまで全工程

アピールポイント

GameManager設計

シングルトンパターンによる統一的なゲーム進行管理

Player物理挙動

Kinematic Rigidbodyで物理演算に頼らない独自の重力・操作感を実現

SoundManager

BGM・SE・ゲームオーバー音を独立管理する音響システム

Pipes / Spawner管理

ランダム高度で生成し、画面外で自動破棄するメモリ効率の良い設計

技術ハイライト

Player : 重力($gravity=-9.8f$)とジャンプ力($strength=5f$)を自作実装し、Unity標準の物理演算に頼らず軽量・調整しやすい操作感を実現

Pipes / Spawner : InvokeRepeatingで一定間隔ごとに生成、Random.Rangeで高さ決定($y:-1f\sim 3.5f$)。画面外に出た瞬間Destroyでメモリを即時解放

Parallax : MeshRendererのmainTextureOffsetを毎フレーム加算するだけで、追加オブジェクトなしの無限スクロール背景を軽量に実現

パイプの生成～破棄ライフサイクル

生成
Instantiate

→

左へ移動

→

画面外(LeftEdge)判定

→

Destroy()
メモリ即解放

開発時間目安 : 約35～40時間（6～7週間・週3～4回×2時間）

GitHub: github.com/Yoshinko45/Flying-Bird

動画: youtube.com/watch?v=XVvTXHVSffM

※開発期間はプログラム説明資料では「2025年7～8月」、ポートフォリオサイトでは「2025年8～9月」と1か月の差異あり（本資料はPDF側の表記に統一）。